TuHospedaje

PLAN DE PRUEBAS

Plan y Reporte de Pruebas de Software — Sprint 1

TuHospedaje — Estructura Base y Catálogo

TC-01 a TC-11, todos aprobados
CASOS

37 tests de backend (29 base + 8 de mapeo)
AUTOMATIZADAS

Equipo de Desarrollo · Mayo 2026

Plan y Reporte de Pruebas de Software — Sprint 1

Proyecto: TuHospedaje **Sprint:** 1 — Base del Sistema **Alcance:** 11 User Stories (US #1 a US #11) **Tipos de prueba:** API (JUnit 5 + MockMvc + Testcontainers), UI Manual, Validaciones Backend

TC-01: Header funcional (US #1)

Campo	Detalle
User Story	US #1 — Encabezado con logo, lema y botones
Precondición	Aplicación frontend corriendo en `http://localhost:5173`
Tipo	UI Manual

#	Paso	Resultado Esperado
1	Cargar `http://localhost:5173`	El header se renderiza en la parte superior
2	Hacer scroll vertical	El header permanece fijo (sticky)
3	Verificar logo	Isologotipo de TuHospedaje visible a la izquierda
4	Verificar lema	Texto "Encuentra tu lugar ideal al mejor precio" visible
5	Clic en el logo	Redirige al Home (`/`)
6	Verificar botón "Iniciar sesión"	Visible a la derecha, estilizado, sin funcionalidad
7	Verificar botón "Crear cuenta"	Visible a la derecha, estilizado como `btn-secondary`, sin funcionalidad
8	Reducir ventana a 480px	Header se adapta sin desbordamientos
9	Reducir ventana a 768px	Elementos se reordenan fluidamente

TC-02: Home — buscador, categorías y recomendaciones (US #2)

Campo	Detalle
Precondición	Backend corriendo con al menos 1 lodging, frontend en `:5173`
Tipo	UI Manual + API

#	Paso	Resultado Esperado
1	Cargar Home	Sección de buscador visible con fondo `var(--primary)`
2	Verificar campos del buscador	Input de destino + 2 inputs de fecha visibles
3	Verificar botón "Buscar"	Visible, no ejecuta acción (UI estática)
4	Verificar sección "Categorías"	Título visible + placeholder "(Sprint 2 — ...)"
5	Verificar sección "Recomendaciones"	Título visible + grilla con alojamientos cargados desde API
6	Verificar que las cards tienen datos reales	Nombre, ciudad y país coinciden con datos de BD
7	Verificar color de fondo del Main	Fondo blanco/claro debajo del buscador
8	Verificar alto del Main	Ocupa al menos el alto de la ventana (`min-height: 100vh`)

TC-03: Registro de alojamiento — POST (US #3)

Nota — numeración "US #3" ambigua entre documentos: este US #3 es el registro básico de alojamiento (sin precio ni capacidad, ver body de referencia más abajo). Una etiqueta distinta, también "US #3", se usó en el commit `d2b44f7` ("feat(lodging): require price per night and max guests") para un cambio posterior fuera de la numeración de esta serie de sprints — no confundir con este US #3 ni con Sprint 3, que documenta `pricePerNight` / `maxGuests` como campos agregados a `Lodging` en esa etapa (ver `sprint-3-report.md` , sección de modelo de datos).

Campo	Detalle
Precondición	Backend corriendo
Tipo	API (Postman)

#	Paso	Resultado Esperado
1	POST `/api/lodgings` con body válido	HTTP 201, body JSON con `id` generado
2	POST con nombre duplicado	HTTP 500 con mensaje "Ya existe un alojamiento con el nombre: ..."
3	POST con email duplicado	HTTP 500 con mensaje "Ya existe un alojamiento con el email: ..."
4	POST con `id` en el body	HTTP 400 (Bad Request)

Body válido de referencia:

```
{
  "name": "Hotel Test QA",
  "description": "Alojamiento de prueba",
  "address": "Calle Test 123",
  "city": "Ciudad Test",
  "country": "Argentina",
  "phoneNumber": "+54119999999",
  "email": "testqa@test.com"
}
```

TC-04: Grilla aleatoria (US #4)

Campo	Detalle
Precondición	Backend con al menos 1 lodging
Tipo	API (Postman) + UI Manual

#	Paso	Resultado Esperado
1	GET `/api/lodgings/random`	HTTP 200, array JSON
2	Verificar máximo 10 elementos	`length <= 10`
3	Repetir request 3 veces	Cada respuesta tiene orden aleatorio
4	Verificar UI en Home	Las cards se muestran en grilla 4 columnas
5	Verificar sin repeticiones	Ningún alojamiento aparece duplicado en la misma respuesta

TC-05: Vista detalle de alojamiento (US #5)

Campo	Detalle
Precondición	Backend con al menos 1 lodging
Tipo	API + UI Manual

#	Paso	Resultado Esperado
1	GET `/api/lodgings/{id}` con ID válido	HTTP 200, objeto JSON con name, description, city, country
2	GET con ID inexistente (ej. 99999)	HTTP 404
3	UI: navegar a `/lodgings/{id}`	Hero/header ocupa el 100% del ancho de la página
4	UI: verificar título	Nombre del alojamiento visible a la izquierda del header
5	UI: verificar flecha de retorno	Botón "←" visible a la derecha del título
6	UI: clic en flecha	Navega a la página anterior
7	UI: verificar descripción	Sección "Descripción" con texto del backend
8	UI: verificar imagen destacada	Imagen principal visible en el hero
9	UI: verificar galería de imágenes	Las imágenes adicionales (si existen) se muestran en grilla

TC-06: Galería de imágenes (US #6)

Campo	Detalle
Precondición	Lodging con al menos 5 `imageUrls` pobladas
Tipo	UI Manual

#	Paso	Resultado Esperado
1	Cargar detalle de alojamiento con 5 imágenes	Layout 50/50: 1 img grande izq + grilla 2×2 der
2	Verificar imágenes en grilla derecha	4 imágenes en formato 2 filas × 2 columnas
3	Cargar alojamiento con 6+ imágenes	Botón "Ver más" visible en esquina inferior derecha
4	Clic en "Ver más"	Se despliegan las imágenes restantes
5	Clic en "Ver menos"	Se ocultan las imágenes adicionales
6	Cargar alojamiento con 0 imágenes	Galería no se renderiza (no rompe la UI)

Nota — cobertura automatizada agregada posteriormente: `frontend/src/pages/ProductDetail/ProductDetail.test.jsx` — `describe('ProductDetail - GalleryModal', ...)` cubre ahora la apertura del modal con la imagen real (`src` / `alt`) y la navegación entre miniaturas, complementando esta verificación manual (commit `2bced9d`).

TC-07: Footer (US #7)

Campo	Detalle
Tipo	UI Manual

#	Paso	Resultado Esperado
1	Cargar cualquier página	Footer visible al final del scroll
2	Verificar isologotipo	Logo de TuHospedaje presente
3	Verificar copyright	Texto "© 2026 TuHospedaje. Todos los derechos reservados."
4	Verificar enlaces de redes	Íconos de Facebook e Instagram visibles
5	Reducir ventana a 480px	Footer se adapta sin desbordamientos

Nota — cobertura automatizada agregada posteriormente:
`frontend/src/components/Footer/Footer.test.jsx` (archivo nuevo) verifica el alt del logo, el texto de copyright y los hrefs de los enlaces a Facebook/Instagram, complementando esta verificación manual (commit `2bced9d`).

TC-08: Paginación del catálogo (US #8)

Campo	Detalle
Precondición	Backend con 11+ lodgings
Tipo	API + UI Manual

#	Paso	Resultado Esperado
1	GET `/api/lodgings` sin parámetros	HTTP 200, array JSON (lista completa)
2	GET `/api/lodgings?page=0&size=10`	HTTP 200, objeto JSON con `lodgings`, `currentPage`, `totalPages`, `totalItems`
3	Verificar `totalItems`	Coincide con cantidad total de lodgings en BD
4	GET `/api/lodgings?page=1&size=10`	Página 2, máximo 10 elementos
5	UI: navegar a `/admin`	Tabla con 10 elementos máximo
6	UI: botón "Inicio"	Deshabilita en página 1, lleva a página 0
7	UI: botón "Anterior"	Deshabilita en página 1, retrocede 1 página
8	UI: botón "Siguiente"	Deshabilita en última página, avanza 1 página
9	UI: contador de página	Muestra "Página X de Y" correcto

TC-09: Panel de Administración (US #9)

Campo	Detalle
Tipo	UI Manual

#	Paso	Resultado Esperado
1	Navegar a `/admin`	Página del panel cargada
2	Verificar menú	Botón "Lista de alojamientos" visible y activo
3	Verificar "Agregar alojamiento"	Botón `+ Agregar alojamiento` visible
4	Abrir modal de creación	Clic en "Agregar alojamiento" muestra formulario emergente
5	Acceder desde celular (DevTools modo móvil)	Cartel "Funcionalidad no disponible para dispositivos móviles"
6	Acceder desde tablet (modo portrait en DevTools)	Mismo cartel de bloqueo

TC-10: Tabla de inventario (US #10)

Campo	Detalle
Precondición	Al menos 1 lodging en BD
Tipo	UI Manual + API

#	Paso	Resultado Esperado
1	Cargar `/admin`	Tabla con columnas: ID, Nombre, Acciones
2	Verificar datos de fila	ID y Nombre coinciden con BD
3	GET `/api/lodgings/{id}` de un ítem de la tabla	Datos consistentes con lo mostrado en UI

TC-11: Eliminar alojamiento (US #11)

Campo	Detalle
Precondición	Al menos 1 lodging en BD
Tipo	API + UI Manual

#	Paso	Resultado Esperado
1	UI: clic en "Eliminar" en una fila	Aparece diálogo de confirmación
2	UI: clic en "Cancelar" en el diálogo	No se elimina, tabla sin cambios
3	UI: clic en "Aceptar" en el diálogo	Lodging eliminado, tabla se refresca
4	Verificar eliminación real	GET `/api/lodgings/{id}` devuelve 404
5	DELETE `/api/lodgings/{id}` con ID inexistente	HTTP 500 con ResourceNotFoundException
6	DELETE `/api/lodgings/{id}` con ID existente	HTTP 200, mensaje "Alojamiento eliminado con ID: X"

Resumen de Ejecución

TC	User Story	Tipo de Prueba	Estado
TC-01	US #1 — Header	UI Manual	✔ Aprobado
TC-02	US #2 — Home	UI Manual + API	✔ Aprobado
TC-03	US #3 — Registro (POST)	API Automatizada	✔ Aprobado
TC-04	US #4 — Aleatorios	API + UI Manual	✔ Aprobado
TC-05	US #5 — Detalle	API + UI Manual	✔ Aprobado
TC-06	US #6 — Galería	UI Manual	✔ Aprobado
TC-07	US #7 — Footer	UI Manual	✔ Aprobado
TC-08	US #8 — Paginación	API + UI Manual	✔ Aprobado
TC-09	US #9 — Panel Admin	UI Manual	✔ Aprobado
TC-10	US #10 — Tabla	UI Manual + API	✔ Aprobado
TC-11	US #11 — Eliminar	API + UI Manual	✔ Aprobado

Leyenda: ✔ Aprobado | ⚠ Aprobado con observaciones | ✘ Fallido | ▢ Pendiente

Métricas Consolidadas de Aseguramiento de Calidad (QA)

- **Batería de Pruebas Automatizadas (Suite JUnit 5 + MockMvc + Testcontainers):** 29 casos de prueba automatizados para la funcionalidad base de Lodging (CRUD, búsqueda, paginación, aleatorios), más 8 tests de mapeo de DTO y entidades. Total 37 tests correspondientes al alcance del Sprint 1 (integrados en la suite general de 105 tests del proyecto).
- **Cobertura de Funcionalidad Crítica:** `LodgingService.save()`, `update()`, `delete()`, `findAll()`, `findById()`, `findByName()`, `findAllRandom()` y `findAllPaginated()` cubiertos con tests unitarios (Mockito). Endpoints HTTP correspondientes cubiertos con tests de integración (MockMvc + Testcontainers).
- **Pruebas de Seguridad Regresivas:** Los endpoints de mutación (`POST`, `PUT`, `DELETE`) se verificaron con tokens de usuario `ROLE_USER` y sin autenticación, validando que devuelven `403 Forbidden` según la protección `@PreAuthorize` agregada en Sprint 2.
- **Pruebas UI Manuales:** La funcionalidad visual (header sticky, galería con "Ver más"/"Ver menos", paginación en UI, bloqueo de admin en móvil) se mantiene como verificación manual documentada.